

Spring Boot

Spring Data JPA

Spring Boot Data-JPA

- Spring Boot data effettuerà la configurazione automatica del `javax.sql.DataSource` se nel classpath riesce a trovare un driver supportato. (es: Connector MySQL, H2 ecc..).
- Lo starter `data-jpa` prende il `DataSource` configurato in base al driver trovato nel classpath e lo usa per la configurazione JPA.
- Dobbiamo fornire la configurazione del `DataSource` attraverso delle proprietà nell'`application.properties` (o `.yaml`).

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-data-jpa</artifactId>  
</dependency>
```

Entities

- Tutte le classi che mappano le tabelle del database vanno annotate con `@Entity`
- Con questa annotazione andiamo a segnalare che la classe mappa una specifica tabella del database
- Ogni entities deve essere una classe POJO (plain old java object) ovvero una classe:
 - non final
 - non abstract
 - con un costruttore vuoto
 - variabili private
 - metodi getter e setter

Entities



```
@Entity
public class Ordine {
    <<parametri>>
    <<Costruttore vuoto>>
    <<Getter e Setter>>
}
```



Entities

- Tutte le Entities devono avere una primary key mappata con @id
- Possiamo aggiungere la tipologia di generazione della primary key tramite l'annotazione @GeneratedValue(strategy = GenerationType.{{type}})
- I tipi sono:
 - AUTO
 - Indica che il provider di persistenza deve scegliere una strategia appropriata per il database specifico
 - IDENTITY
 - Indica che il provider di persistenza deve assegnare chiavi primarie per l'entità utilizzando una colonna di identità del database
 - SEQUENCE
 - Indica che il provider di persistenza deve assegnare chiavi primarie per l'entità utilizzando una sequenza di database
 - TABLE
 - Indica che il provider di persistenza deve assegnare chiavi primarie per l'entità utilizzando una tabella del database sottostante per garantire l'univocità

Entities



```
@Entity
public class Ordine {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private long id;
    <<Costruttore vuoto>>
    <<Getter e Setter>>

}
```



Entities

- Tutte le altre colonne possono essere annotate con `@Column` questo ci permette di definire dei parametri come:
 - `updatable`
 - `unique`
 - `nullable`
 - `insertable`
 - `name`

Entities



```
@Entity
public class Ordine {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private long id;
    @Column(updatable = true, unique = true,
            nullable = true, insertable = true,
            name = "nome_ordine")
    private String nomeOrdine;
    <<Costruttore vuoto>>
    <<Getter e Setter>>
}
```



Entities

- Nel caso di relazione tra più tabelle abbiamo tre tipi di relazioni possibili
- @ManyToOne
- @OneToMany
- @ManyToMany

ManyToOne

- Se la relazione è una `@ManyToOne` avremo all'interno della classe un attributo del tipo della classe con la quale esiste la relazione
- In oltre per utilizzare i parametri della `@Column` l'annotation sarà `@JoinColumn`
- La classe con l'annotation `@ManyToOne` si definisce «proprietaria» della relazione ed è quella che conterrà la foreign- key sul database

ManyToOne



```
@Entity
public class Ordine {
    <<altri parametri>>

    @ManyToOne
    @JoinColumn
    private Utente cliente;
    <<Costruttore vuoto>>
    <<Getter e Setter>>

}
```



OneToMany

- Se la relazione è una `@OneToMany` avremo all'interno della classe un attributo del tipo `Collection<T>` dove T è la classe con la quale esiste la relazione
- La classe con la quale esiste la relazione DEVE avere un attributo del tipo della nostra classe mappato con `@ManyToOne`
- All'interno dell'annotation come parametro inseriremo `mappedBy="nomeDellaVariabileDellAltraClasse"`

OneToMany



```
@Entity
public class Utente {
    <<altri parametri>>

    @OneToMany(mappedBy = "cliente")
    private List<Ordine> ordini;

    <<Costruttore vuoto>>
    <<Getter e Setter>>

}
```



ManyToMany

- Se la relazione è una `@ManyToMany` avremo all'interno della classe un attributo del tipo `Collection<T>` dove T è la classe con la quale esiste la relazione
- Una delle due classi deve diventare "proprietaria" della relazione mappando la tabella di relazione che verrà creata sul db con l'annotation `@JoinTable` nella quale relazioneremo entrambe le foreign-key
- Se vogliamo rendere la relazione bidirezionale l'altra tabella utilizzerà il `mappedBy`

ManyToMany

```
@Entity
public class Ordine {
    <<altri parametri>>

    @ManyToMany
    @JoinTable(name = "prodotti_acquistati",
                joinColumns = @JoinColumn(name="id_ordine"),
                inverseJoinColumns = @JoinColumn(name="id_prodotto"))
    private List<Prodotto> prodotti;

    <<Costruttore vuoto>>
    <<Getter e Setter>>

}
```

FetchType

- Una cosa che possiamo definire su tutte le tipologie di relazioni tra entità è il fetchType,
- Questo va a definire quando scaricare gli attributi dell'oggetto relazionato
- esistono solo due tipi di fetch:
 - LAZY
 - Scarica gli attributi solo se viene chiamato il getter dell'attributo
 - EAGER
 - Scarica gli attributi appena viene scaricato l'oggetto dal database

CascadeType

- Un'ulteriore cosa che possiamo definire su tutte le tipologie di relazioni tra entità è il `cascadeType`,
- Questo va a definire come si comporterà con gli attributi relazionati quando effettueremo operazioni sull'oggetto selezionato
- esistono vari tipi di cascase che sono:
 - ALL
 - Propaga tutte le operazioni
 - PERSIST
 - Propaga solo le operazioni di persistenza
 - MERGE
 - Propaga le operazioni di «merging»
 - REMOVE
 - Propaga le operazioni di rimozione
 - REFRESH
 - propaga il ricaricamento dell'entità dal db ogni volta che verrà effettuato per l'entità "padre"
 - DETACH
 - Propaga anche agli attributi la rimozione dal persistence context

ManyToMany

```
@Entity
public class Ordine {
    <<altri parametri>>

    @ManyToMany(cascade = CascadeType.ALL, fetch = FetchType.EAGER)
    @JoinTable(name = "prodotti_acquistati",
                joinColumns = @JoinColumn(name = "id_ordine"),
                inverseJoinColumns = @JoinColumn(name = "id_prodotto"))
    private Set<Prodotto> prodotti;

    <<Costruttore vuoto>>
    <<Getter e Setter>>

}
```

Spring Boot Data-JPA

- Spring Boot data effettuerà la configurazione automatica del `jakarta.sql.DataSource` se nel classpath riesce a trovare un driver supportato. (es: Connector MySQL, H2 ecc..).
- Lo starter `data-jpa` prende il `DataSource` configurato in base al driver trovato nel classpath e lo usa per la configurazione JPA.
- Dobbiamo fornire la configurazione del `DataSource` attraverso delle proprietà nell'`application.properties` (o `.yml`).

Application.properties

- Una volta configurate tutte le nostre classi dovremo aggiungere i parametri all'interno dell'application properties
- I parametri da aggiungere sono
 - L'url del database
 - Lo username di connessione al db
 - La password di connessione al db
 - Il driver class name utilizzato per connettere il database
- Altri parametri che possiamo aggiungere sono
 - Se vogliamo visualizzare in console le query eseguite
 - La tipologia di ddl da eseguire sul db

Application.properties

```
spring.datasource.url = jdbc:mysql://localhost:3306/ordini
```

```
spring.datasource.username = root
```

```
spring.datasource.password = toor
```

```
spring.datasource.driver-class-name = com.mysql.cj.jdbc.Driver
```

```
spring.jpa.show-sql = true
```

```
spring.jpa.hibernate.ddl-auto = update
```

Application.yml

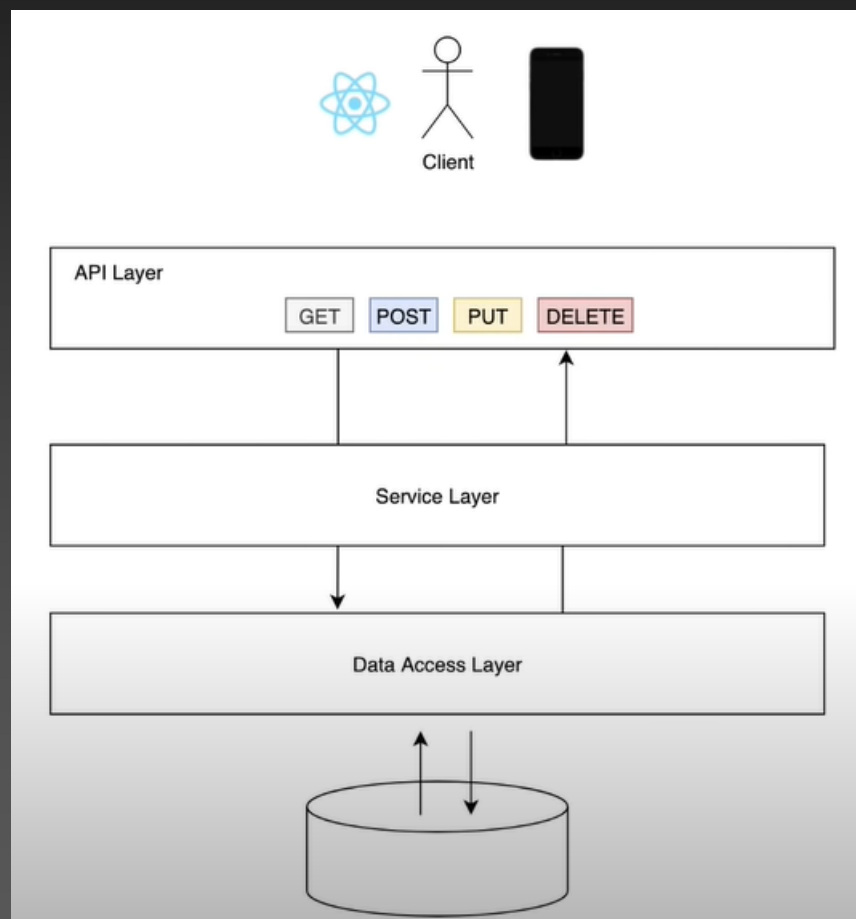
```
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/ordini
    driver-class-name: com.mysql.cj.jdbc.Driver
    username: root
    password: toor

  jpa:
    show-sql: true
    hibernate:
      ddl-auto: update
```

Spring Boot

Architettura a Layer

Architettura a layer



Repository layer

- Non serve annotare con `@Repository` la nostra interfaccia.
- L'interfaccia `JpaRepository<T, ID>` incapsula un'annotazione chiamata `@NoRepositoryBean`. Questa annotazione comunica a spring di non creare bean del tipo `JpaRepository`, Ma di creare i bean delle classi che IMPLEMENTANO la nostra interfaccia. Tali bean sono creati in fase di esecuzione.
- Responsabilità:
 - Interagire con la sorgente dati.
 - Un repository per ogni entity.

Repository layer



```
public interface OrdineRepository extends JpaRepository<Ordine, Long> {  
    @Query(nativeQuery = true, value = "select * from Ordine where  
data_inizio >= :dataInizio and data_fine <= :dataFine")  
    List<Ordine> trovaPerData(LocalDate dataInizio, LocalDate dataFine);  
  
    @Query("select o from Ordine o where o.id = :id")  
    Optional<Ordine> trovaPerId(Long id);  
  
    List<Ordine> findAllByCliente_IdAndDisattivatoIsFalse(Long id);  
  
}
```



Service layer

- Annotazione @Service
- Responsabilità:
 - Contengono la business logic dell'applicazione.
- Caratteristiche:
 - Invocare metodi del repository.
 - Validazione.
 - Conversione da entity a DTO e viceversa (DTO pattern).

Service layer

```
public interface OrdineService {  
    Ordine findById(long id);  
    List<Ordine> findAll();  
}
```

```
@Service  
public class OrdineServiceImpl implements OrdineService{  
    @Autowired  
    private OrdineRepository repo;  
  
    @Override  
    public Ordine findById(long id) {  
        return repo.findById(id).orElseThrow(RuntimeException::new);  
    }  
  
    @Override  
    public List<Ordine> findAll() {  
        return repo.findAll();  
    }  
}
```

Controller layer

- Annotazione @RestController
- Responsabilità:
 - Accettare le richieste HTTP
 - Invocare la logica aziendale.
 - Rispondere in base a ciò che è stato processato.
 - Prima linea contro utenti non autorizzati ad accedere ai service.
 - Gestire eccezioni propagate dagli altri layer.

Controller layer

```
@RestController
public class OrdineController {

    private final OrdineService service;

    public OrdineController(OrdineService service) {
        this.service=service;
    }

    @GetMapping("/ordine/{id}")
    public ResponseEntity<Ordine> findOrdine(@PathVariable("id") long id){
        Ordine o=service.findById(id);
        if(o==null)return ResponseEntity.status(HttpStatus.BAD_REQUEST).build();
        else return ResponseEntity.ok(o);
    }

    @GetMapping("/ordini")
    public ResponseEntity<List<Ordine>> listaOrdini(){
        return ResponseEntity.ok(service.findAll());
    }

}
```

Esercizio



Creare una applicazione web che permetta la gestione di un singolo autonoleggio

Sono previsti, attualmente, due ruoli:

Amministratore di sistema

Cliente che effettua il noleggio

L'autonoleggio possiede soltanto automobili

I veicoli si dividono in categorie (segmenti), ad esempio utilitarie, SUV, berline, ecc.

Ogni categoria ha una tariffa giornaliera, una tariffa settimanale e una tariffa mensile.

L'amministratore:

Aggiungere, modificare e visualizzare le categorie

Aggiungere, rimuovere, modificare e visualizzare i veicoli

